

Exercises 09

Transaction Processing: Concurrency, Recovery

[\[Show with no answers\]](#) [\[Show with all answers\]](#)

1. Consider the following transaction $T1: R(X), R(X)$

- a. Give an example of another transaction $T2$ that, if run concurrently to transaction $T1$ without some form of concurrency control, could interfere with $T1$ to produce unrepeatable reads. Show the sequence of operations which would cause the problem.

[\[show answer\]](#)

- b. Explain how the application of strict two-phase locking would prevent the problem described in your previous answer.

[\[show answer\]](#)

2. SQL supports four isolation-levels and two access-modes, for a total of eight combinations of isolation-level and access-mode. Each combination implicitly defines a class of transactions; the following questions refer to these eight classes:

- a. Describe which of the following phenomena can occur at each of the four SQL isolation levels: dirty read, unrepeatable read, phantom problem.

[\[show answer\]](#)

- b. Why does the access mode of a transaction matter?

[\[show answer\]](#)

3. Draw the precedence graph for the following schedule:

T1:	R(A) W(Z)		C
T2:		R(B) W(Y)	C
T3:	W(A)		W(B) C

[\[show answer\]](#)

4. [Based on RG Ex.17.2] Consider the following incomplete schedule S:



T1:	R(X)	R(Y)	W(X)	W(X)
T2:		R(Y)		R(Y)
T3:			W(Y)	

a. Determine (by using a precedence graph) whether the schedule is serializable

[\[show answer\]](#)

b. Modify S to create a complete schedule that is conflict-serializable

[\[show answer\]](#)

5. Is the following schedule conflict serializable? Show your working.

T1:		R(X)	W(X)	W(Z)		R(Y)	W(Y)
T2:	R(Y)	W(Y)	R(Y)		W(Y)	R(X)	W(X) R(V) W(V)

[\[show answer\]](#)

6. [Based on RG Ex.17.3] For each of the following schedules, state whether it is conflict-serializable and/or view-serializable. If you cannot decide whether a schedule belongs to either class, explain briefly. The actions are listed in the order they are scheduled, and prefixed with the transaction name.

- T1:R(X) T2:R(X) T1:W(X) T2:W(X)
- T1:W(X) T2:R(Y) T1:R(Y) T2:R(X)
- T1:R(X) T2:R(Y) T3:W(X) T2:R(X) T1:R(Y)
- T1:R(X) T1:R(Y) T1:W(X) T2:R(Y) T3:W(Y) T1:W(X) T2:R(Y)
- T1:R(X) T2:W(X) T1:W(X) T3:W(X)

[\[show answer\]](#)

7. Recoverability and serializability are both important properties of concurrent transaction schedules. They are also orthogonal. Serializability requires that the schedule be equivalent to some serial ordering of the transactions. Recoverability requires that each transaction commits only after all of the transactions from which it has read data have also committed.

Using the following two transactions:

T1:	W(A)	W(B)	C	T2:	W(A)	R(B)	C
-----	------	------	---	-----	------	------	---

give examples of schedules that are:



- a. recoverable and serializable
- b. recoverable and not serializable
- c. not recoverable and serializable

[\[show answer\]](#)

8. ACR schedules avoid the potential cascading rollbacks that can make recoverable schedules less than desirable. Using the transactions from the previous question, give an example of an ACR schedule.

[\[show answer\]](#)

9. Consider the following two transactions:

T1	T2
-----	-----
read(A)	read(B)
A := 10*A+4	B := 2*B+3
write(A)	write(B)
read(B)	read(A)
B := 3*B	A := 100-A
write(B)	write(A)

a. Write versions of the above two transactions that use two-phase locking.

[\[show answer\]](#)

b. Is there a non-serial schedule for $T1$ and $T2$ that is serializable? Why?

[\[show answer\]](#)

c. Can a schedule for $T1$ and $T2$ result in deadlock? If so, give an example schedule. If not, explain why not.

[\[show answer\]](#)

10. What is the difference between quiescent and non-quiescent checkpointing? Why is quiescent checkpointing not used in practice?

[\[show answer\]](#)

11. Consider the following sequence of undo/redo log records:

<START T> ; <T,A,10,11> ; <T,B,20,21> ; <COMMIT T>



Give all of the sequences of “events” that are legal according to the rules of undo/redo logging. An “event” consists of one of: writing to disk a block containing a given data item, and writing to disk an individual log record.

[\[show answer\]](#)

12. Consider the following sequence of undo/redo log records from two transactions T and U:

```
<START T> ; <T,A,10,11> ; <START U> ; <U,B,20,21> ;  
<T,C,30,31> ; <U,D,40,41> ; <COMMIT U> ; <T,E,50,51>;  
<COMMIT T>
```

Describe the actions of the recovery manager, if there is a crash and the last log record to appear on disk is:

- a. **<START U>**
- b. **<COMMIT U>**
- c. **<T,E,50,51>**
- d. **<COMMIT T>**

You may assume that there is an **<END CKPT>** record in the log immediately before the start of transaction T, so that we do not need to worry about the log any further back than the start of T.

[\[show answer\]](#)

